

Clasificarea tipurilor de orez

Ganciu Emma

Dataset: rice_data_type.csv

1. Descrierea datasetului si a problemei	2
2. Verificarea datelor si decizii de preprocesare	2
Parametri definiti	3
Decodificarea coloanei tinta	3
Integritatea datelor	3
Distributia datelor	3
Outlieri	4
Scalarea datelor	4
3. Descrierea experimentelor	5
Impartirea datelor	5
Cross-validation si tuning	5
Resampling	6
4. Comparatii intre parametri pe kNN	6
Influenta dimensiunii setului de test (test_size)	6
Influenta seed-ului (random_state)	7
Comparatia intre metrice	7
Concluzie	7
5. Comparatii intre clasificatori	8
Observatii	8
Analiza kNN in functie de k	8
Concluzie	9
Interpretarea metricilor si analiza erorilor	9
Matricea de confuzie	10
Explicatii TN TP FN FP	10
Analiza erorilor	11
Concluzii si limitari	12
Concluzii generale	12
Limitari	12
Pasi urmatari propusi	12

1.Descrierea datasetului si a problemei

Datasetul analizat contine un total de 3810 instante si 8 coloane, dintre care 7 reprezinta attribute numerice, iar una reprezinta eticheta (clasa).

Atributele sunt:

- arie (Area)
- perimetru (Perimeter)
- lungimea axei mari (Major_Axis_Length)
- lungimea axei mici (Minor_Axis_Length)
- excentricitate (Eccentricity)
- aria convexa (Convex_Area)
- extindere (Extent)

Variabila tinta este „Class”, care poate lua doua valori: Cammeo si Osmancik.

Problema abordata este una de clasificare binara, scopul fiind construirea unui model capabil sa diferentieze intre cele doua tipuri de orez pe baza caracteristicilor sale.

2.Verificarea datelor si decizii de preprocesare

Pentru organizarea mai buna a experimentelor si pentru asigurarea reproductibilitatii rezultatelor, s-a utilizat un fisier de configurare (config.py). Acesta contine parametri constanti folositi pe parcursul intregului proces de procesare si antrenare.

- fisierul de configurare are mai multe avantaje importante:
- centralizeaza parametrii importanti ai experimentelor
- permite modificarea usoara a setarilor fara a altera logica principala
- asigura consistenta intre rulari diferite ale programului

Parametri definiti

1. SEED (RANDOM_STATE) – valoare fixa utilizata pentru initializarea generatorului de numere aleatoare
2. DEFAULT_TEST_SIZE - proportia setului de test
3. TARGET_COLUMN - numele coloanei tinta

Un aspect esential in experimentele de machine learning este controlul randomizarii. Operatii precum impartirea datelor in train/test, initializarea ponderilor (pentru retele neuronale), selectia subseturilor (in anumite algoritmi) depind de procese aleatoare.

Prin setarea unei valori fixe pentru SEED(RANDOM_STATE) se obtin urmatoarele beneficii:

- reproductibilitate: aceleasi rezultate pot fi obtinute la fiecare rulare;
- comparabilitate: modelele pot fi comparate corect intre ele, deoarece folosesc aceeasi impartire a datelor;
- debugging mai usor: erorile pot fi reproduse si analizate consistent.

Decodificarea coloanei tinta

Variabila tinta avea byte encoding (format b'Cammeo'), si a fost transformata intr-un format string ("Cammeo"), necesar pentru antrenarea modelelor.

Integritatea datelor

Nu exista valori lipsa in dataset, ceea ce simplifica semnificativ etapa de preprocesare.

Distributia datelor

Distributia claselor arata un usor dezechilibru:

- Osmancik: 2180 instante (~57.22%)
- Cammeo: 1630 instante (~42.78%)

Raportul intre clase este aproximativ 1.34, ceea ce indica un dezechilibru moderat, dar nu critic. Acest aspect trebuie totusi luat in considerare in evaluarea modelelor, mai ales pentru metrice precum recall sau f1-score.

Din punct de vedere statistic, se observa diferente clare intre cele doua clase:

- Cammeo are valori medii mai mari pentru Area, Perimeter si Major_Axis_Length.
- Osmancik are valori mai mici in general, dar o variabilitate similara.

Aceste diferente sugereaza ca problema este bine separabila, cel putin partial, in spatiul caracteristicilor.

Analiza tipurilor de date arata ca toate variabilele de intrare sunt de tip float64, iar variabila tinta este de tip obiect (categorica).

Outlieri

Detectarea outlierilor folosind metoda IQR a relevat:

- Un numar foarte mic de outlieri pentru Area (4), Convex_Area (3)
- Mai multi pentru Minor_Axis_Length (65, ~1.71%)
- Cateva valori pentru Eccentricity (21)

Procentul total de outlieri este redus, astfel incat nu s-a considerat necesara eliminarea acestora.

Scalarea datelor

S-au testat doua metode de scalare:

- StandardScaler
- MinMaxScaler

Rezultatele arata:

- Accuracy cu StandardScaler: 0.909
- Accuracy cu MinMaxScaler: 0.915

MinMaxScaler a oferit performanta usor mai buna, ceea ce sugereaza ca modelele sensibile la scala (precum kNN) beneficiaza de normalizare.

3. Descrierea experimentelor

Impartirea datelor

S-au realizat mai multe experimente cu diferite valori pentru:

- test_size: 0.2, 0.4, 0.7
- seed: 0, 5, 42

Rezultatele sunt relativ stabile:

- Accuracy variaza intre ~0.866 si ~0.895
- F1 variaza intre ~0.860 si ~0.891

Aceasta stabilitate indica faptul ca modelul nu este extrem de sensibil la impartirea datelor.

Cross-validation si tuning

```
kNN GridSearchCV - cel mai bun scor (accuracy): 0.885
kNN GridSearchCV - cei mai buni parametri: {'n_neighbors': 51}
kNN GridSearchCV - accuracy pe test cu modelul ales: 0.866
Decision Tree GridSearchCV - cel mai bun scor (accuracy): 0.929
Decision Tree GridSearchCV - cei mai buni parametri: {'max_depth': 15, 'min_samples_leaf': 100}
Decision Tree GridSearchCV - accuracy pe test cu modelul ales: 0.916
```

Pentru optimizarea hiperparametrilor s-a utilizat GridSearchCV pentru a verifica cel mai bun parametru dintre cei oferiti:

- Pentru kNN: n_neighbors = 51
- Pentru Decision Tree: max_depth = 15, min_samples_leaf = 100

Rezultate:

- kNN: scor CV = 0.885, test accuracy = **0.866**
- Decision Tree: scor CV = 0.929, test accuracy = **0.916**

Decision Tree a beneficiat semnificativ de tuning, depasind clar celelalte modele.

Resampling

Nu s-a aplicat resampling, deoarece dezechilibrul nu este sever. Totusi, pentru imbunatatiri viitoare s-ar putea testa tehnici precum SMOTE.

4. Comparatii intre parametri pe kNN

Rezultatele obtinute pentru diferite modele:

Test Size	Seed	Accuracy	Precision	Recall	F1
0.2	0	0.895	0.903	0.885	0.891
0.4	0	0.887	0.893	0.877	0.883
0.7	0	0.885	0.888	0.876	0.880
0.2	5	0.886	0.892	0.876	0.881
0.4	5	0.889	0.887	0.886	0.886
0.7	5	0.878	0.876	0.874	0.875
0.2	42	0.866	0.875	0.854	0.860
0.4	42	0.869	0.875	0.858	0.863
0.7	42	0.876	0.884	0.865	0.871

Influenta dimensiunii setului de test (test_size)

Se observa ca:

Pentru test_size = 0.2, modelul obtine in general cele mai bune rezultate (accuracy pana la 0.895).

Pe masura ce test_size creste la 0.4 si 0.7, performanta scade usor.

Explicatie:

Cand setul de antrenare este mai mare (test_size mic), modelul are mai multe date pentru invatare, ceea ce duce la performanta mai buna.

Cand setul de test devine foarte mare (0.7), setul de antrenare este redus, iar modelul nu mai invata la fel de eficient.

Totusi, diferentele nu sunt foarte mari, ceea ce indica un model relativ stabil.

Influenta seed-ului (random_state)

Seed-ul controleaza modul in care sunt impartite datele. Observam:

Pentru seed = 0, performanta este cea mai buna si constanta.

Pentru seed = 5, rezultatele sunt similare, cu mici variatii.

Pentru seed = 42, performanta este usor mai scazuta.

Explicatie:

Diferite valori ale seed-ului duc la diferite impartiri ale datelor.

Unele impartiri pot fi mai favorabile (distributie mai echilibrata intre train si test), altele mai dificile.

Faptul ca diferentele sunt mici arata ca modelul nu este foarte sensibil la aceasta variatie.

Comparatia intre metrice

Precision este in general mai mare decat recall → modelul face mai putine false positive.

Recall este usor mai mic → unele instante pozitive nu sunt detectate.

F1-score este echilibrat (~0.86 – 0.89), ceea ce indica o performanta generala buna.

Rezultatele arata ca performanta modelului kNN este foarte stabila indiferent de valoarea lui k.

Accuracy variaza foarte putin (aproximativ intre 0.866 si 0.869), ceea ce indica faptul ca problema nu este foarte sensibila la alegerea acestui hiperparametru.

Se observa ca:

Cele mai bune rezultate (accuracy si F1) apar pentru $k = 151$ si $k = 501$, dar diferenta este minima.

Pentru valori foarte mari ale lui k (1523), precision creste usor, dar recall scade, ceea ce inseamna ca modelul devine mai conservator.

Valorile mici ale lui k (15) ofera rezultate comparabile, fara instabilitate majora.

Concluzie

Modelul este stabil, deoarece variatiile intre diferite configuratii sunt mici;

Performanta este consistenta indiferent de seed sau split;

5.Comparatii intre clasificatori

Exemplu pe accuracy intre modele:

Model	Accuracy
knn	0.869
Decision Tree	0.898
Gaussian NB	0.891
MLP	0.572

De ce se folosesc aceste modele:

- kNN este simplu, fara presupuneri (parametri); lent la predictie, sensibil la scara feature-urilor si la zgomot.
- Decision Tree este usor de interpretat
- GaussianNB este rapid, probabilistic;
- MLP este flexibil, non-liniar; cere de obicei scalare si tuning; cu setari default poate merge slab fata de celelalte modele.

Observatii

- Decision Tree este cel mai performant model
- Gaussian NB are rezultate bune, dar sub Decision Tree
- kNN este decent, dar inferior
- MLP performeaza slab (probabil subantrenat sau parametri nepotriviti)

Analiza kNN in functie de k

Nota:

Folosim macro pentru metrice: (average="macro"): este util cand vrem sa vedem daca modelul e potrivit si pe clase rare, nu doar pe cea majoritara. Ambele tipuri de boabe contează la fel (nu vreau sa optimizez doar pentru clasa mai frecventa).

La un dezechilibru atât de mic cum avem pe datasetul nostru, macro și weighted vor fi apropiate, deci nu schimbă dramatic rezultatul.

k	Accuracy	Precision	Recall	F1
15	0.867	0.874	0.856	0.862
51	0.866	0.875	0.854	0.860
151	0.869	0.875	0.857	0.863
501	0.869	0.875	0.857	0.863
1523	0.867	0.880	0.853	0.861

Concluzie

Performanta este relativ stabila, ceea ce indica faptul ca problema nu este extrem de sensibila la alegerea lui k.

Interpretarea metricilor si analiza erorilor

Pentru unul dintre modele:

- Accuracy: 0.869
- Precision: 0.875
- Recall: 0.857
- F1: 0.863

Aceste valori indica un echilibru bun intre precizie si acoperire.

Matricea de confuzie

254	72
28	408

Interpretare:

- 254 Cammeo corect clasificate
- 72 Cammeo clasificate gresit ca Osmancik
- 28 Osmancik clasificate gresit ca Cammeo
- 408 Osmancik corect clasificate

Modelul face mai multe erori pentru clasa Cammeo, ceea ce poate fi explicat prin:

- suprapunerea caracteristicilor
- dezechilibrul usor al claselor

Explicatii TN TP FN FP

- | | | |
|------------------------|--|-----|
| 1. TN (True Negative) | Real negativ, prezis negativ | 254 |
| 2. FP (False Positive) | Real negativ, prezis pozitiv (fals alarmă) | 72 |
| 3. FN (False Negative) | Real pozitiv, prezis negativ (ratat) | 28 |
| 4. TP (True Positive) | Real pozitiv, prezis pozitiv | 408 |

Pe scurt:

- TP / TN = modelul a ghicit bine (pozitiv/pozitiv sau negativ/negativ).
- FP = luat ca si pozitiv, dar era negativ.
- FN = luat ca si negativ, dar era pozitiv.

Avem:

- multe predictii corecte (254 + 408)
- mai multe FP decat FN (72 vs 28), deci modelul tinde sa supraestimeze clasa pozitiva (mai des zice „pozitiv” cand nu e).

Se confunda cel mai des Cammeo (luat drept Osmancik): 72 cazuri din 326 (254+72) Cammeo reale -> $72/326 = 0,221$ (~**22,1% erori**).

Invers, 28 Osmancik luate drept Cammeo din 436(408+28) -> $28/436 \approx 0,064$ (~**6,4%**).

Eroare totala pe tot testul: $100/762 \approx 0,131$ (~13,1%).

Analiza erorilor

Am analizat 3 exemple de bob Osmancik clasificat in mod gresit drept Cammeo:

Real: Osmancik, Prezis: Cammeo cu index in dataset: 3632 Attribute: Area 13586.000000 Perimeter 467.023987 Major_Axis_Length 191.151123 Minor_Axis_Length 91.863765 Eccentricity 0.877007 Convex_Area 13888.000000 Extent 0.798707	Real: Osmancik, Prezis: Cammeo cu index in dataset: 1924 Attribute: Area 13818.000000 Perimeter 467.895996 Major_Axis_Length 189.284760 Minor_Axis_Length 94.479927 Eccentricity 0.866520 Convex_Area 14142.000000 Extent 0.617454	Real: Osmancik, Prezis: Cammeo cu index in dataset: 3396 Attribute: Area 13619.000000 Perimeter 467.765991 Major_Axis_Length 193.680939 Minor_Axis_Length 90.473480 Eccentricity 0.884191 Convex_Area 13926.000000 Extent 0.586672
--	--	--

Analizand attributele bobului de tip Cammeo comparativ cu bobul de tip Osmancik

Attribute Cammeo:							
	Area	Perimeter	Major_Axis_Length	Minor_Axis_Length	Eccentricity	Convex_Area	Extent
count	1304.000000	1304.000000	1304.000000	1304.000000	1304.000000	1304.000000	1304.000000
mean	14182.992331	487.872159	205.601773	88.856094	0.900942	14516.976227	0.650769
std	1252.292855	21.664448	10.162169	5.275343	0.013555	1274.552484	0.081917
min	9908.000000	410.506012	170.781647	67.695343	0.837433	10205.000000	0.498075
25%	13336.500000	473.923752	198.901539	85.500957	0.893267	13653.500000	0.580142
50%	14233.500000	488.847992	205.858047	88.876675	0.901552	14571.500000	0.633824
75%	14988.000000	502.871262	212.435818	92.143549	0.909832	15361.000000	0.716344
max	18913.000000	548.445984	239.010498	107.542450	0.948007	19099.000000	0.861050
Attribute Osmancik:							
	Area	Perimeter	Major_Axis_Length	Minor_Axis_Length	Eccentricity	Convex_Area	Extent
count	1744.000000	1744.000000	1744.000000	1744.000000	1744.000000	1744.000000	1744.000000
mean	11553.939220	429.328119	176.173093	84.563684	0.875829	11803.755734	0.671172
std	1047.112916	20.248714	9.401941	5.319619	0.019092	1068.727663	0.072442
min	7551.000000	359.100006	145.264465	59.532406	0.777233	7723.000000	0.516425
25%	10861.500000	416.134254	169.837551	81.434492	0.862989	11107.750000	0.611309
50%	11560.500000	429.239502	175.536308	84.738674	0.875942	11815.500000	0.654427
75%	12267.000000	442.520241	181.987736	88.038574	0.888874	12519.250000	0.735274
max	15420.000000	503.459991	206.056549	101.762260	0.931275	15800.000000	0.832747

Putem vedea de ce exemplele au fost clasificate gresit.

Boabele Osmancik clasificate gresit au

- Area ~13600–13800
- Perimeter ~467
- Eccentricity ~0.87–0.88

adica valori situate mai aproape de valorile Cammeo, ceea ce confirma ca erorile apar in zonele de suprapunere.

Concluzii si limitari

Concluzii generale

- Datasetul este bine structurat si fara valori lipsa
- Exista o separare rezonabila intre clase
- Decision Tree este cel mai performant model (accuracy ~0.916 dupa tuning)
- Scalarea imbunatateste performanta, in special pentru kNN
- Modelele sunt stabile fata de diferite splituri ale datelor

Limitari

- Dezechilibrul claselor poate influenta usor performanta
- Outlierii nu au fost tratati explicit
- MLP nu a fost optimizat corespunzator
- Nu s-au folosit tehnici avansate de feature engineering

Pasi urmasori propusi

1. Simularea unui dezechilibru si aplicarea unor metode de resampling (SMOTE, undersampling)
2. Optimizarea mai avansata a MLP (numar de straturi, neuroni, rata de invatare)
3. Testarea altor algoritmi
4. Analiza importantei caracteristicilor pentru clasificarea bobului de orez
5. Validare mai robusta folosind k-fold cross-validation